

# Chosen Architecture

The system uses a unified **Audio-LLM** architecture that combines a speech encoder with a Large Language Model (LLM). This replaces traditional multi-stage pipelines with three distinct, modular components:

- Audio Encoder: Processes raw acoustic data (converted to log-mel spectrograms) to extract dense audio features, capturing acoustic properties, prosody, and distinct speaker characteristics.
- Adapter: Convert audio features to a vector (1d array information that the LLM can use)
- Language Model: Decodes the features into proper text since this is a language model

The benefits of using this system are as followed:

- Modularity: Each component are plug and play. The audio encoder can be swapped or fine-tuned for noisy environments without retraining the underlying language model backbone.
- Native code switching: depending on the LLM usage it can be used to handle multilingual and code switching scenarios

## Model Selection

The model i have selected would be Qwen 3 ASR which comes in 2 flavours

- 0.6B
- 1.7B

And at the same time comes with a forced aligner, so it returns the timestamps, though its a separate model (0.6B) from the main ASR model compared to nemo where its all in one solution.

The benefits are as followed:

- Multilingual / Code switching support ( Mandarin, English, Malay, Indonesian)
- Inference and Training does not require very high end hardware (3060 is good enough)
- Code Feasibility is easy to read and edit this makes maintainability a lot easier

## Chunking / Windowing Strategy

Based on the official Qwen3-ASR framework, the streaming pipeline captures audio in **500ms chunks** and holds them in **1.0-second blocks** across a **4-chunk buffer** to allow the model to dynamically rewrite and correct text using 1 to 1.5 seconds of live context. To manage hardware constraints and prevent the GPU from running out of memory during long conversations, the system enforces a strict **30-to-60-second session limit**. Once this cap is reached, the backend

flushes the remaining text, clears the GPU's memory cache, and instantly initiates a fresh session seamlessly behind the scenes.

## Handling of partial transcripts

Due to the nature of the model using an LLM the decoding process can be used to predict complex linguistic text. After fine tuning on the local language it allows the model to further adapt it to the local languages context and wording.

## Handling of code switching

Due to the nature of the task at hand the model has been pretrained on the languages provided ahead of time as well as further finetune on code switching task in 1 sentence therefore handles it very well

The finetuned model further exhibits handling slang of the regional accent such as (e.g walao eh)

## Latency bottlenecks

The absolute largest latency bottleneck is the physical VRAM limit of the RTX 3060 when handling encoding and decoding lengths simultaneously. To prevent a growing context window from completely saturating the card's 12GB memory bus and crashing the system, the model is restricted to a strict max token length of 4096 for decoding. While wrapping the pipeline in a persistent WebSocket streaming endpoint significantly reduces immediate compute loads by serving small 500ms chunks at a time, the compounding data still hits a physical memory ceiling over time, requiring a hard 30-to-60-second limit per session to purge the memory cache and keep the AI running smoothly without clogging the hardware.

## Optimization Strategies

We can optimise our inference strategy by using the following tactics

- vLLM and Paged Attention
  - This allows the Key-Value (KV) cache of the LLM to be managed and accessed similarly to standard system RAM. By storing the continuous audio stream context in dynamic, non-contiguous virtual pages, the system avoids memory-copy delays and eliminates the need to recalculate past audio features, making real-time streaming inference drastically faster and more efficient.
- Quantisation
  - Using bits and bytes to convert to 4bit or 8bit

- To allow a bigger batch size to be imputed or allow a large max token size to be decoded
- Memory Resets / Session
  - Once an active user reaches our strict 30-to-60-second limit, the backend executes a hard state reset. This action instantly drops the session keys, clears the corresponding entries from the PagedAttention memory pages.

## Production Deployment Considerations

- Model Storage & Versioning
  - Using S3 as a place to store the model and pull when necessary this allows the model to be changed anytime
  - Adding versioning to the code / data or model allows it to be reverted or upgraded with a specific numbering in mind
- Docker & Kubernetes (Code Scalability)
  - Wrap the code down into a docker image so that it can be reused continuously
  - Further deploy and scale it with kubernetes to allow scalability (e.g replica)
- Prometheus & Grafana (Monitoring)
  - We can track it since everything is wrapped in kubernetes therefore it's easy to keep track of certain metrics like how many failed cases average speed of cases all within a dashboard